
OWQ: Lessons learned from activation outliers for weight quantization in large language models

Changhun Lee^{1*} Jungyu Jin^{1*} Taesu Kim² Hyungjun Kim² Eunhyeok Park¹

¹Pohang University of Science and Technology, POSTECH ²SqueezeBits Inc.
{changhun.lee, jgjin0317}@postech.ac.kr,
{taesu.kim, hyungjun.kim}@squeezebits.com, canusglow@gmail.com

Abstract

Large language models (LLMs) with hundreds of billions of parameters show impressive results across various language tasks using simple prompt tuning and few-shot examples, without the need for task-specific fine-tuning. However, their enormous size requires multiple server-grade GPUs even for inference, creating a significant cost barrier. To address this limitation, we introduce a novel post-training quantization method for weights with minimal quality degradation. While activation outliers are known to be problematic in activation quantization, our theoretical analysis suggests that we can identify factors contributing to weight quantization errors by considering activation outliers. We propose an innovative PTQ scheme called outlier-aware weight quantization (OWQ), which identifies vulnerable weights and allocates high-precision to them. Our extensive experiments demonstrate that the 3.01-bit models produced by OWQ exhibit comparable quality to the 4-bit models generated by OPTQ.

1 Introduction

Large language models (LLMs) [2, 26, 28, 29, 35] demonstrate impressive generation performance on a wide range of complex language tasks solely relying on prompt tuning and few-shot examples without the need for task-specific fine-tuning. This suggests a potential increase in LLM applications in the future. However, the memory and computational demands of LLMs present significant challenges, not just for training but for inference as well. For instance, the GPT3-175B model requires about 350GB of space for storing model parameters. More than 5 A100 GPUs having 80GB of memory are required for inference, demanding hundreds of thousands of dollars to build. This expensive cost hampers the widespread adoption of LLMs.

Weight quantization is an attractive optimization method for large language models (LLMs) as it can significantly reduce system requirements. By storing parameters with low precision through quantization, storage space can be considerably saved. This also leads to various performance benefits, including increased ALU utilization and reduced communication costs by allocating more operations to a single GPU [24]. Furthermore, quantization can address memory bottlenecks that cause performance degradation, particularly in single-batch inference scenarios with low data reuse. Performance improvement is also expected through low-precision acceleration. Advanced studies [10, 24] have shown that matrix multiplication with 3-bit weight and fp16 activation exhibits remarkable performance improvements on a single GPU compared to matrix multiplication with fp16 weights and activation using 5 GPUs. Thus, weight quantization of LLMs is crucial as it can offer practical performance enhancements.

*Equal Contribution.

However, weight quantization of LLMs presents a trade-off, as it can degrade the output quality of the model. Minimizing this quality loss is vital for its widespread adoption, but LLM quantization is challenging due to the need for a fast quantization process considering the large size of LLMs and the requirement to avoid task-specific fine-tuning to preserve zero-shot generative performance. In advance, OPTQ [10] (or known as GPTQ [9]) has introduced layer-wise post-training quantization (PTQ) based on optimal brain compression (OBC) algorithm [8]. This approach allows for the quantization of the largest OPT [35] model in approximately 4 hours, with minimal impact on accuracy, even at 3-bit precision. The model’s capacity can be compressed to less than a quarter of the size of the existing fp16 format, making it accessible even with a single A100 GPU. This work demonstrates the potential of extreme weight quantization for LLMs.

In this paper, we introduce a new weight quantization technique, called Outlier-aware Weight Quantization (OWQ), an advancement over OPTQ that significantly enhances the quality of the quantized model. Previous studies [4, 32, 34] have identified outliers in certain activation feature dimensions. These outliers, due to their diverse data range, complicate the quantization process of activation. While our work utilizes fp16 for activation, our analysis indicates that activation outlier plays a key role in amplifying quantization-induced errors in weight. OWQ uses a mixed-precision quantization scheme, which applies higher precision to the weights susceptible to quantization caused by activation outliers. Our extensive analysis shows that the 3.01-bit OWQ model shows comparable quality to the 4-bit OPTQ model. To the best of our knowledge, we are the first study to incorporate the existence of activation outlier in extreme weight quantization. Our key contributions can be summarized accordingly:

- We introduce a novel analysis that reveals how activation outliers can amplify the error in weight quantization, providing valuable insights for improving the quality of quantized weights.
- Building upon this analysis, we propose a new weight quantization algorithm called OWQ. OWQ not only preserves the benefits of low-precision but also significantly reduces the quantization error.
- Through extensive analysis, we demonstrate that OWQ achieves comparable performance with 3.01-bit to that of OPTQ with 4-bit across diverse LLM tasks without incurring notable performance overhead.

2 Background and Related Works

2.1 Quantization and LLMs

Quantization is a widely-used optimization technique aimed at exploiting the benefit of low-precision while maintaining the quality of the target network. While the primary advantage is the reduction in storage space, it is worth noting that quantization also provides substantial performance improvement through the support of low-precision acceleration. The main drawback of quantization is the degradation of output quality. Mitigating quality degradation is highly important, and early studies focused on quantization-aware training (QAT) [7, 12, 36], which tried to restore quality through additional training. However, as the understanding of quantization grew and various techniques emerged, post-training quantization (PTQ) [15, 20, 19, 31] have been actively studied, enabling quality preservation without training.

Due to the LLM’s necessity of significant storage space and computational resources, it is crucial to apply optimization via quantization. Numerous studies have been conducted, but the distinctive characteristics of LLMs steer the research in a unique direction. In general, QAT has been favored for extremely low-bit precision to minimize quantization error. However, it is less favorable for LLM quantization because of the high cost of the training environment. Moreover, QAT might not be the optimal choice to preserve the task-agnostic generation performance of LLMs.

Instead, PTQ has emerged as an important topic for LLM quantization. This field has two distinct approaches: one aims to quantize both activations and weights to int8 [4, 34], considering both capacity reduction and performance enhancement. In contrast, the second approach focuses solely on weight quantization to sub-4-bit precision [10, 24]. In this paper, we align our work with the latter approach. While concentrating on weight quantization, we devise a novel quantization scheme, drawing significant inspiration from int8-related research on activation quantization.

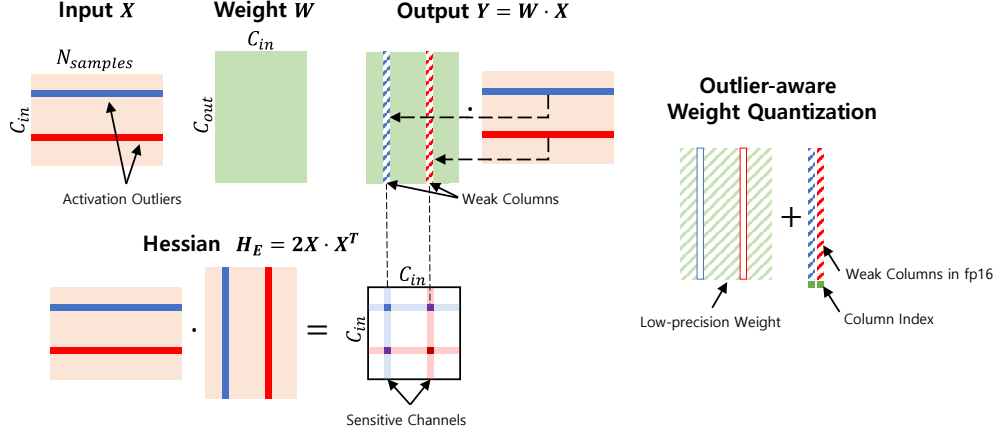


Figure 1: Overview of the proposed idea, outlier-aware weight quantization.

2.2 Int8 Quantization for Activation and Weight

Int8 multiplication can provide up to 2x performance improvements and more than 5x energy consumption reduction compared to fp16 baselines [13]. Numerous studies [4, 34] aim to quantize both activation and weight to int8 for matrix multiplication operations in LLMs. However, those studies identify a unique challenge of LLMs for activation quantization. **LLMs exhibit a few outliers in intermediate activations, with values significantly larger than other activations, and these outliers are concentrated in specific feature dimensions. Preserving the values of these outliers is known to be crucial for maintaining accuracy after activation quantization.**

In this study, while activation remains at full-precision and only weight quantization is applied, we figure out that the presence of activation outliers still impacts the sensitivity of weight quantization. We also demonstrate that considering activation outliers is essential for accurate weight quantization.

2.3 OPTQ: Weight Quantization for LLMs

OPTQ [10] represents the cutting-edge research in the field of weight quantization for LLMs. It is based on Optimal Brain Compression (OBC) [8], which employs element-wise quantization (pruning) and compensation, using a Hessian-based metric of layer-wise quantization errors (Eq. (1)). This approach differs from previous studies, which applied quantization through gradient descent based on the straight-through estimator [15, 31] or a rounding-to-nearest mechanism [4].

$$w_q = \underset{w_q}{\operatorname{argmin}} \frac{(\operatorname{quant}(w_q) - w_q)^2}{[H_F^{-1}]_{qq}}, \quad \delta_F = -\frac{w_q - \operatorname{quant}(w_q)}{[H_F^{-1}]_{qq}} \cdot (H_F^{-1})_{:,q} \quad (1)$$

OPTQ has optimized OBC to apply quantization in parallel for each element of the input dimension, facilitating rapid quantization and demonstrating remarkable performance even at 3-bit. However, if the model size decreases or the problem’s complexity increases, the accuracy still falls compared to the fp16 baselines. **In this paper, we suggest selectively applying high-precision to weights that are vulnerable to quantization caused by activation outliers, while applying OPTQ to the remaining weights. These enhancements can significantly reduce the quantization error while preserving the quantization performance of OPTQ.**

3 Problem Definition and Motivation

In this section, prior to introducing our idea, we first aim to clearly define the problem and explain how our findings can address it. This paper aims to apply the layer-wise linear quantization for the weights of LLMs with minimal quality degradation. When an input feature $X \in \mathbb{R}^{C_{in} \times N}$ is given, where C_{in} represents the number of input channels and N is the sequence length of the input,

the full-precision weight matrix $W \in \mathbb{R}^{C_{out} \times C_{in}}$ for C_{out} output features is iteratively updated to minimize the difference between the output activations before and after quantization. The objective function to find the quantized weight $\hat{W}$ that minimizes the squared error is defined as follows:

$$\arg \min_{\hat{W}} E = \arg \min_{\hat{W}} \|WX - \hat{W}X\|_2^2 \quad \text{s.t.} \quad C(\hat{W}) < C_t, \quad (2)$$

where $C(\cdot)$ represents the compression ratio and C_t is the target compression ratio. Solving the optimization problem represented by Eq. (2) is known to be a challenging task, as it falls under the category of NP-hard problems [10, 19]. In our work, we leverage the OPTQ algorithm for weight quantization. However, it is worth noting that other linear quantization algorithms can also be applied to address this problem. The layer-wise quantization process is applied sequentially from the model input to the output, ensuring the comprehensive quantization of all weights in the model.

3.1 Layer-wise Linear Quantization and Hessian of Weights

In this subsection, we drive the relationship between the sensitivity of weights and the activation outliers in terms of quantization. Similar to the approach used in OBC, we reorganize the squared error term in Eq. (2) as the sum of squared errors for each output channel in the weight matrix, and the equation is modified as $\sum_{i=1}^{C_{out}} \|W_{i,:}X - \hat{W}_{i,:}X\|_2^2$. Through this decomposition, we can clearly observe that the overall error is separated into individual errors for each output channel.

With the modified equation, our focus shifts to two key aspects. Firstly, it is important to note that there is no Hessian interaction between output channels. Specifically, the individual Hessians with respect to the layer-wise quantization error, denoted as $H_i \in \mathbb{R}^{C_{in} \times C_{in}}$, have an identical value as:

$$H_i = \frac{\partial^2 E}{\partial W_{i,:}^2} = 2XX^T. \quad (3)$$

Secondly, as observed in previous studies [19], the individual error term can be approximated using Taylor expansion. By setting $\Delta W_{i,:} = W_{i,:} - \hat{W}_{i,:}$, the error for the i -th output channel can be expressed as follows:

$$E_i = \|W_{i,:}X - \hat{W}_{i,:}X\|_2^2 \approx \Delta W_{i,:}H_i\Delta W_{i,:}^T. \quad (4)$$

For the detailed proof, please refer to the supplementary materials. The use of the Hessian as a metric to measure sensitivity in quantization has been widely adopted in various quantization approaches [5, 6]. As these studies have pointed out, particularly in the context of layer-wise quantization, the output error can be directly related to the Hessian of the weights.

Keeping these two observations in mind, we can derive a surprising insight by acknowledging the presence of activation outliers in LLMs. Previous studies [4, 34] have reported that certain feature dimensions of LLM activation contain outliers with significantly larger values than others. These activation outliers cause some elements of H_i to exhibit exceptionally high values, as illustrated in Figure 1. This abnormal surge in Hessian values increases the channels' sensitivity to quantization. As indicated in Eq. (4), even when the same degree of weight perturbation is present, the ensuing change in output can be considerably larger due to some large elements of H_i . Therefore, if an activation outlier exists, a perturbation in the connected weight will result in a larger output error. We refer to the weights that are susceptible to quantization, specifically those associated with the activation outliers in a specific input channel, as a weak column (shown in Figure 1).

Hence, if we quantize all weights to the same bit-width during the weight quantization process, the quantization error at the weak columns corresponding to the activation outliers can lead to substantial perturbation on the output, resulting in a significant quantization error. To minimize this quality degradation, it is crucial to apply special handling for those weak columns.

4 OWQ: Outlier-aware Weight Quantization

To address this problem, we propose a novel idea called Outlier-aware Weight Quantization (OWQ). The algorithm is designed on top of OPTQ, but it includes additional pipelines that selectively handle weak columns with high-precision, minimizing quality degradation.

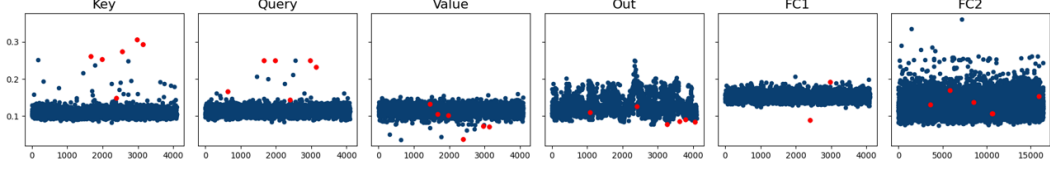


Figure 2: Min-max range of the weights in each column (blue dots) and the selected weak columns among those (red dots).

In the previous section, we highlighted the relationship between the Hessian matrix and sensitivity caused by activation outliers in Eq. (3). We also demonstrated that the final error is influenced by the quadratic terms of perturbations with the Hessian matrix in Eq. (4). In addition, a previous study [5] has suggested using the product of the trace of the Hessian matrix and the Frobenius norm of the perturbation to approximate the sensitivity of quantization. Building on these insights, we define the sensitivity of j -th column as follows:

$$\text{sensitivity}_j = \lambda_j \|\Delta W_{:,j}\|_2^2, \quad (5)$$

where λ_j is the j -th diagonal element of the Hessian matrix. In this work, we extend the study [5] that allocates layer-wise bit-width based on each layer’s sensitivity: the sensitivity is measured at a more granular level, focusing on the columns of the weight matrix. By analyzing the sensitivity of individual columns, we can effectively identify the weak columns that are vulnerable to quantization and require higher precision. When the goal is to select a specific number (k) of weak columns, the proposed metric is utilized to choose the top- k columns based on their highest sensitivity values.

Moreover, as depicted in Figure 2, it’s important to note that our approach is distinctly different from existing outlier-aware quantization studies [22, 23]. In those studies, outlier weights are excluded from quantization based solely on their magnitude, with the motivation to minimize the error of weights before and after the quantization. However, our method focuses on not only minimizing the quantization error for the weights but also that of the output activation. Therefore, we utilize a Hessian metric related to the output of matrix multiplication, and the weights are selected based on their sensitivity, rather than its magnitude, as shown in the figure.

After identifying the weak columns, we store those vectors with high precision. In practice, we store a complete low-precision matrix with zero-filled weak columns. Additionally, we store the weak columns as fp16 (16-bit floating-point) and use an extra single 16-bit integer per column, which represents the column index of the weak columns. If we denote the matrix multiplication operation by $\times$, the output of the matrix multiplication can be calculated as the sum of (zero-filled quantized weight matrix $\times$ fp16 activation matrix) and (fp16 weak columns $\times$ corresponding fp16 activation channels). The additional storage overhead is solely caused by the weak columns. This overhead is negligible ($\approx 0.3\%$, demonstrated in Sec. 5), while the accuracy is significantly improved. In addition, the remaining weights are quantized using OPTQ, but we introduce a key advancement to minimize the quantization error even further, as explained in the following subsection.

4.1 Quantization Hyperparameter Tuning

Following the selection of weak columns, the remaining weights are quantized using the OPTQ method. Since OPTQ also employs sequential column-wise quantization, the exclusion of weak columns can be seamlessly integrated into the OPTQ framework.

However, we have made a modification to the existing OPTQ, which applied min-max quantization. In our experiments, we tuned the quantization hyperparameters, such as step size and zero point, to narrow the quantization range than the maximum and minimum value range of the weight. Previous studies reported that truncation of data led to a significant reduction in quantization error by balancing truncation and rounding errors [7, 15, 21, 31], but performance degradation was observed in OPTQ, hence they use a naive min-max range. However, in our experiments, after removing the weak columns, the introduction of truncation significantly helped in reducing the quantization error and producing reliable output. According to our empirical observations, the weak columns in the key and query layers of transformer blocks have exceptionally large values. Truncating these values led to a very large error, resulting in a significant loss of accuracy. However, in our method, this error does not occur as the weak column are maintained with full-precision.

In our experiments, we employ a simple quantization method based on rounding to the nearest with truncation to adjust the quantization parameters. The optimal values of the parameters are searched greedily to minimize the difference between the weights before and after quantization. After searching for the best quantization parameters, we apply OPTQ on top of the searched values.

5 Experiments

5.1 Experimental Setup

To validate the outstanding performance of our proposed method, we present quantization results for large-scale LLMs such as OPT [35], LLaMA [29], and BLOOM [28] (Section C.1) families. Our primary baseline is OPTQ, so we apply identical experimental settings of it. For instance, our calibration dataset consists of 128 random 2048 token segments from the C4 dataset [27]. All experiments were conducted on a single NVIDIA A100 GPU with 80GB of main memory. Like OPTQ, our method quantizes the target model without re-training. The reported results are obtained in a zero-shot environment. To measure the zero-shot performance, we utilize an open-source evaluation repository, EleutherAI/lm-evaluation-harness [11]. Please note that we report the numbers with an error margin based on 5 experiments with different seeds.

Compared to the baselines of 3-bit and 4-bit OPTQ results, we present two variants, with an extra 0.01 bit and 0.1 bit overhead, respectively. The additional storage area of extra bit is evenly distributed across the linear layers within the transformer block. For instance, in the OPT model which has six linear layers (key, query, value, out, fc1, and fc2), the weak columns of the key layer contribute an average of 0.00167 bit in the 3.01-bit configuration (0.125% columns of the weight matrix). This extra bit covers the all overhead of the mixed-precision representation. If we quantize OPT-175B model with an average of 3.01 bits, it will require approximately 220 MB of additional storage compared to the 3-bit OPT-175B OPTQ model, which utilizes around 65.6 GB of storage space. All experiments were conducted using the PyTorch 2.0 [25] framework with HuggingFace integration [33], and the source code is available at <https://github.com/xvyaward/owq>.

5.2 Results of Perplexity Measure

The accuracy of the proposed model is assessed through the evaluation on multiple language tasks, including WikiText-2 [17], Penn Treebank (PTB) [16], and C4. Perplexity-based tasks are particularly sensitive to model quantization [10], with perplexity numbers serving as indicators of the generative performance of the quantized model. The results for WikiText-2 can be found in Table 1 and Table 2, while the results for PTB and C4 are provided in the supplementary material.

Table 1: OPT WikiText-2 perplexity (lower is better). For the results with *, we used 3.05 bits; there are few or no weak columns in the budget of 3.01 bits due to the small model dimension.

OPT	Bits	125M	350M	1.3B	2.7B	6.7B	13B	30B	66B
full	16	27.65	22.00	14.63	12.47	10.86	10.13	9.56	9.34
RTN	4	37.28	25.94	48.17	16.92	12.10	11.32	10.98	110
OPTQ	4	31.43	24.21	15.56	12.82	11.41	10.34	9.57	9.55
OWQ	4.01	29.55$\pm_{.23}$	23.05$\pm_{.12}$	14.97$\pm_{.06}$	12.36$\pm_{.02}$	10.89$\pm_{.02}$	10.27$\pm_{.01}$	9.52$\pm_{.00}$	9.23$\pm_{.03}$
OPTQ	3	53.05	34.45	21.17	16.83	15.09	11.73	10.3	14.42
OWQ	3.01	35.19$\pm_{.46}$*	26.54$\pm_{.27}$*	16.37$\pm_{.07}$	13.19$\pm_{.08}$	11.22$\pm_{.06}$	11.51$\pm_{.05}$	9.58$\pm_{.01}$	9.32$\pm_{.03}$
OWQ	3.1	33.46$\pm_{.35}$	26.01$\pm_{.17}$	15.35$\pm_{.17}$	12.92$\pm_{.12}$	11.17$\pm_{.04}$	10.38$\pm_{.04}$	9.58$\pm_{.05}$	9.31$\pm_{.04}$

Table 2: LLaMA WikiText-2 perplexity

LLaMA	Bits	7B	13B	30B	65B
full	16	5.68	5.09	4.10	3.53
RTN	4	6.29	5.53	4.54	3.92
OPTQ	4	6.84	5.34	4.45	3.87
OWQ	4.01	5.94$\pm_{.02}$	5.25$\pm_{.00}$	4.24$\pm_{.01}$	3.73$\pm_{.00}$
OPTQ	3	18.07	6.76	5.84	5.15
OWQ	3.01	6.65$\pm_{.02}$	5.65$\pm_{.03}$	4.76$\pm_{.02}$	4.22$\pm_{.01}$
OWQ	3.1	6.39$\pm_{.02}$	5.57$\pm_{.02}$	4.63$\pm_{.01}$	4.08$\pm_{.01}$

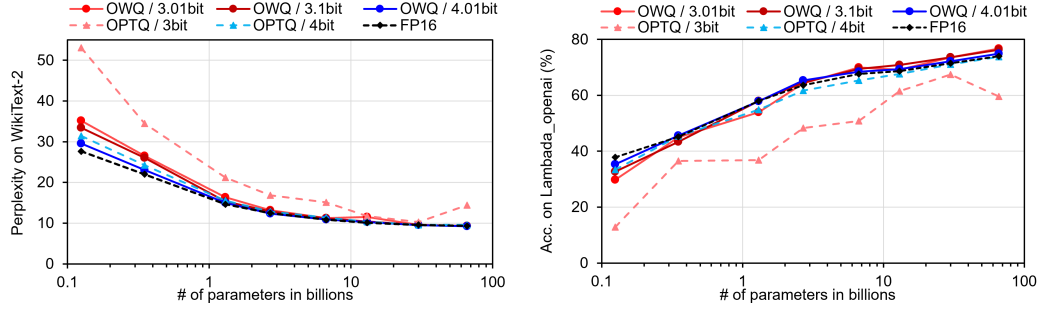


Figure 3: OPT family perplexity (left) and zero-shot accuracy (right).

The results on the tables clearly demonstrate that OWQ consistently delivers substantial quality improvements across the LLM families, irrespective of the model size. The 3.01-bit model effectively mitigates the quality degradation observed in the 3-bit OPTQ model, while the 3.1-bit model achieves comparable performance to the 4-bit OPTQ model. Furthermore, OWQ 4.01-bit yields noteworthy improvements, highlighting the significance of treating weak columns. These results underscore the importance and effectiveness of our approach in preserving model quality after quantization.

An interesting finding is the significant improvement in model quality for models with less than 13 billion parameters when applying OWQ. Although previous studies have highlighted the presence of activation outliers in models with more than 6.7 billion parameters, even smaller models with moderately large channels can still benefit from mixed precision quantization. This suggests that the concept of weak columns remains valid and effective in enhancing the quality of LLMs, regardless of the model size.

5.3 Results of Various Zero-shot Tasks

We conducted additional experiments on diverse zero-shot language tasks. The results are available at Table 3 and Table 4. Our approach shows significant performance improvements across all tested tasks. Even our models using 3.01 bits clearly outperform the 4-bit RTN, and exhibit results on par with the 4-bit of OPTQ models. Our method’s strength lies in its universal applicability, consistently boosting the performance of generative models with minimal storage overhead. The Pareto-front plot in Figure 3 clearly illustrates the advantages of OWQ compared to OPTQ.

Table 3: OPT lambada_openai zero-shot accuracy (%) (higher is better). For the results with *, we used 3.05 bits; there are few or no weak columns in the budget of 3.01 bits (small model dimension).

OPT	Bits	125M	350M	1.3B	2.7B	6.7B	13B	30B	66B
full	16	37.84	45.18	58.01	63.57	67.63	68.64	71.45	73.98
RTN	4	22.87	23.89	29.52	31.23	34.56	35.75	38.05	8.40
OPTQ	4	33.38	45.41	54.81	61.74	65.28	67.60	71.11	73.75
OWQ	4.01	35.29$\pm$.26	45.59$\pm$.53	57.86$\pm$.58	65.32$\pm$.32	68.47$\pm$.19	69.34$\pm$.11	72.18$\pm$.02	74.88$\pm$.08
OPTQ	3	12.87	36.53	36.81	48.28	50.78	61.51	67.46	59.61
OWQ	3.01	29.78$\pm$1.15	45.00$\pm$.97	53.94$\pm$.88	64.36$\pm$.56	69.94$\pm$.45	69.13$\pm$.41	73.58$\pm$.18	76.76$\pm$.21
OWQ	3.1	32.66$\pm$.89	43.41$\pm$1.39	57.93$\pm$.98	64.54$\pm$.78	69.42$\pm$.37	70.84$\pm$.30	73.61$\pm$.30	76.26$\pm$.20

Table 4: LLaMA winogrande (left) and ARC-challenge (right) zero-shot accuracy (%).

LLaMA	Bits	winogrande				ARC-challenge			
		7B	13B	30B	65B	7B	13B	30B	65B
full	16	67.09	70.09	72.61	77.43	41.38	44.62	45.39	46.33
RTN	4	67.17	68.67	72.38	75.14	39.59	42.32	46.08	47.01
OPTQ	4	63.00	69.42	72.53	75.55	38.52	43.45	45.54	46.80
OWQ	4.01	66.03 $\pm$.18	69.27 $\pm$.20	74.03$\pm$.59	76.24$\pm$.36	39.65$\pm$.05	43.32 $\pm$.22	44.65 $\pm$.82	46.25 $\pm$.09
OPTQ	3	51.52	66.41	69.01	72.22	25.72	39.27	40.70	43.05
OWQ	3.01	64.88$\pm$.88	68.68$\pm$.75	72.26$\pm$.26	73.05$\pm$.93	36.64$\pm$.93	40.73$\pm$.56	45.17$\pm$.50	43.80$\pm$.23
OWQ	3.1	65.87$\pm$.71	69.55$\pm$.37	72.93$\pm$.58	75.17$\pm$.52	39.66$\pm$.58	41.55$\pm$.34	44.39$\pm$.63	44.68$\pm$.59

Table 5: Effective bit-width sweep using OPT-6.7B model.

Effective bit-width	3.005	3.01	3.05	3.10	3.20	3.30
WikiText2	11.38	11.22	11.19	11.17	11.15	11.13
PTB	16.61	16.32	16.29	16.30	16.28	16.27
C4	13.35	13.23	13.19	13.18	13.16	13.16

5.4 Acceleration on Real Device

To show the benefits of low-precision acceleration, we created a customized CUDA kernel for OWQ and measured its latency overhead on an A100 GPU. Notably, we set weak columns in the low-precision matrix to zero, resulting in the low-precision process having the same overhead as OPTQ’s customized kernel. Additionally, we integrated a high-precision operation for weak columns. The activation input channels for these weak columns are selected on-the-fly, and a dense high-precision GeMV kernel is used. In the 3.1-bit model, the mixed process computation increases latency by only 3.79 % compared to the 3-bit acceleration of the OPTQ kernel on OPT-175B model. Considering the benefits gained, the computational overhead is negligible.

5.4.1 Effect of the Extra Bit Size

To analyze the impact of the number of weak columns, we measured the perplexity of the OPT-6.7B model while varying the extra bit ratio (Table 5). While our main focus was on reporting performance using 0.01 and 0.1 extra bits, we observed significant performance improvements even with fewer weak columns. With just 3.005 bits, the model already surpasses the performance of the OPTQ 3-bit model. However, as the bit width increases, the performance gains gradually diminish, reaching a saturation point. It is important to note that activation outliers are limited to only a few dimensions, resulting in a small number of corresponding weak columns. Consequently, even a small number of weak columns can lead to noticeable performance improvements.

5.5 Quantization time cost

The speed of the quantization algorithm is of utmost importance for LLM quantization. OWQ, compared to OPTQ, introduces extra operations for selecting weak columns and tuning quantization hyperparameters. However, by sharing certain expensive items like Hessian with OPTQ, the specific overhead of OWQ can be minimized. Table 6 presents the quantization time over various LLMs. OWQ can still successfully quantize a 66B-scale model in under 3 hours, showcasing its practicality for real-world applications.

Table 6: Algorithm running time for quantization of OPT (left) and LLaMA (right) models.

OPT	6.7B	13B	30B	66B	LLaMA	7B	13B	30B	65B
OPTQ time	11.2m	20.1m	43.9m	1.6h	OPTQ time	9.0m	15.8m	35.9m	1.1h
OWQ time	16.4m	30.8m	68.5m	2.4h	OWQ time	14.4m	26.1m	66.3m	2.0h

5.6 Layer-wise Quantization Sensitivity

In our experiments, we uniformly allocated extra storage space to linear layers, but observed varying sensitivity of weak columns across layers. In the OPT-6.7B model, we assessed layer-by-layer sensitivity by applying OWQ to select layers within a fixed capacity budget. Our results in Table 7 indicate the best accuracy is achieved when weak columns are removed simultaneously from both

Table 7: Layer-wise sensitivity analysis on OPT-6.7B model.

layers	W_k, W_q	W_k, W_v	W_k, W_{fc1}	W_q, W_v	W_v, W_{fc1}
WikiText-2	11.26	19.87	22.25	111.57	89.92
layers	W_k, W_q, W_v	W_k, W_q, W_{fc1}	W_k, W_q, W_{out}	W_q, W_v, W_{out}	
WikiText-2	11.31	11.24	11.26	111.81	

Table 8: The results of OWQ and OPTQ with fine-grained quantization

Method / Bits / Group size	OPT-6.7B					LLaMA-7B				
	Wiki2	PTB	C4	lambada $\uparrow$		Wiki2	PTB	C4	winog. $\uparrow$	arc_c. $\uparrow$
OPTQ / 3	15.09	22.49	17.29	50.78		18.07	260.36	21.77	51.52	25.72
OPTQ / 3 / g128	11.28	16.59	13.32	64.47		8.95	111.35	11.02	57.95	32.63
OWQ / 3.01	11.22	16.32	13.23	69.94		6.65	55.60	8.62	64.88	36.64
OWQ / 3.01 / g1024	11.18	16.32	13.19	68.91		6.59	54.48	8.50	65.19	36.76
OWQ / 3.01 / g128	11.16	16.23	13.10	68.61		6.41	50.30	8.19	66.16	38.14

key and query weights. Performance suffers significantly if either layer isn't addressed, especially for queries alone. OWQ's hyperparameter tuning stage is more sensitive to weak columns in key and query layers with higher magnitudes than other layers. Improvements could be made by allocating space at different rates according to each layer's sensitivity. However, the rate tuning is prohibitively expensive, making implementing this approach challenging. Developing a method to tune layer-wise weak column numbers is a potential future research direction.

6 Discussions

6.1 Fine-grained Quantization

Applying linear quantization at fine-grained granularity significantly reduces quantization error while introducing some storage overhead for quantization hyperparameters. OPTQ utilizes this approach by dividing row vectors into groups (e.g., group size of 128 or 1024) and applying linear quantization independently with different configurations. This expansion can be applied orthogonally to OWQ, so we can combine row-wise fine-grained quantization with OWQ to assess any additional improvements. Results in Table 8 show that the room for improvement from fine-grained quantization is negligible, as OWQ already substantially enhances the 3-bit model's quality. Moreover, compared to grouped OPTQ with 128 group size, 3.01-bit OWQ's storage overhead is only 8% of grouped OPTQ overhead while achieving better perplexity and zero-shot accuracy. Thus, OWQ is a superior solution to the grouping technique.

6.2 Comparison with Act-Ordering of OPTQ

While not discussed in OPTQ papers, the "act-order" option was recently added to their official GitHub [14]. This option, which quantizes based on activation magnitude, differs from the previous OPTQ approach that applied quantization to columns sequentially. Generally, "act-order" produces superior results. Although no theoretical interpretation exists for this method, our study suggests that the benefits of "act-order" arise from sensitivity-aware quantization. This means that applying sequential quantization beginning with sensitive columns improves performance, even within the OPTQ method. However, act-order alone cannot sufficiently mitigate the quality degradation caused by weak columns within a low-precision domain. OWQ, by addressing the weak columns with high-precision, consistently outperforms OPTQ + act order, regardless of act-order option.

7 Conclusion

The presence of activation outliers has been identified as a significant challenge in LLM activation quantization. We found that even in weight quantization, activation outliers can increase the sensitivity of certain weight columns, leading to significant quality degradation in a low-precision domain. To overcome this, we introduced a novel quantization scheme, OWQ. Compared to existing 3-bit quantization methods, OWQ achieves substantial quality improvements with only negligible storage and computation overhead, effectively preserving the benefits of low-precision acceleration. We believe that our theoretical insights will expand the understanding of weight quantization, inspiring future research and promoting the widespread adoption of LLMs.

References

- [1] Yonatan Bisk, Rowan Zellers, Jianfeng Gao, Yejin Choi, et al. Piqa: Reasoning about physical common-sense in natural language. In *Proceedings of the AAAI conference on artificial intelligence*, volume 34, pages 7432–7439, 2020.
- [2] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.
- [3] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv preprint arXiv:1803.05457*, 2018.
- [4] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. Llm.int8(): 8-bit matrix multiplication for transformers at scale. *arXiv preprint arXiv:2208.07339*, 2022.
- [5] Zhen Dong, Zhewei Yao, Daiyaan Arfeen, Amir Gholami, Michael W Mahoney, and Kurt Keutzer. Hawqv2: Hessian aware trace-weighted quantization of neural networks. *Advances in neural information processing systems*, 33:18518–18529, 2020.
- [6] Zhen Dong, Zhewei Yao, Amir Gholami, Michael W Mahoney, and Kurt Keutzer. Hawq: Hessian aware quantization of neural networks with mixed-precision. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 293–302, 2019.
- [7] Steven K Esser, Jeffrey L McKinstry, Deepika Bablani, Rathinakumar Appuswamy, and Dharmendra S Modha. Learned step size quantization. *arXiv preprint arXiv:1902.08153*, 2019.
- [8] Elias Frantar and Dan Alistarh. Optimal brain compression: A framework for accurate post-training quantization and pruning. In *Advances in Neural Information Processing Systems*, 2022.
- [9] Elias Frantar, Saleh Ashkboos, Torsten Hoefer, and Dan Alistarh. Gptq: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323*, 2022.
- [10] Elias Frantar, Saleh Ashkboos, Torsten Hoefer, and Dan Alistarh. Optq: Accurate quantization for generative pre-trained transformers. In *The Eleventh International Conference on Learning Representations*, 2023.
- [11] Leo Gao, Jonathan Tow, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Kyle McDonell, Niklas Muennighoff, Jason Phang, Laria Reynolds, Eric Tang, Anish Thite, Ben Wang, Kevin Wang, and Andy Zou. A framework for few-shot language model evaluation, September 2021.
- [12] Jonathan Green, Tony Charman, Helen McConachie, Catherine Aldred, Vicky Slonims, Pat Howlin, Ann Le Couteur, Kathy Leadbitter, Kristelle Hudry, Sarah Byford, et al. Parent-mediated communication-focused treatment in children with autism (pact): a randomised controlled trial. *The lancet*, 375(9732):2152–2160, 2010.
- [13] Mark Horowitz. 1.1 computing’s energy problem (and what we can do about it). In *2014 IEEE International Solid-State Circuits Conference Digest of Technical Papers (ISSCC)*, pages 10–14. IEEE, 2014.
- [14] IST-DASLab. gptq. <https://github.com/IST-DASLab/gptq>, 2022.
- [15] Yuhang Li, Ruihao Gong, Xu Tan, Yang Yang, Peng Hu, Qi Zhang, Fengwei Yu, Wei Wang, and Shi Gu. Breqq: Pushing the limit of post-training quantization by block reconstruction. In *International Conference on Learning Representations*, 2021.
- [16] Mitch Marcus, Grace Kim, Mary Ann Marcinkiewicz, Robert MacIntyre, Ann Bies, Mark Ferguson, Karen Katz, and Britta Schasberger. The penn treebank: Annotating predicate argument structure. In *Human Language Technology: Proceedings of a Workshop held at Plainsboro, New Jersey, March 8-11, 1994*, 1994.
- [17] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models. *arXiv preprint arXiv:1609.07843*, 2016.
- [18] Todor Mihaylov, Peter Clark, Tushar Khot, and Ashish Sabharwal. Can a suit of armor conduct electricity? a new dataset for open book question answering. *arXiv preprint arXiv:1809.02789*, 2018.
- [19] Markus Nagel, Rana Ali Amjad, Mart Van Baalen, Christos Louizos, and Tijmen Blankevoort. Up or down? adaptive rounding for post-training quantization. In *International Conference on Machine Learning*, pages 7197–7206. PMLR, 2020.
- [20] Markus Nagel, Mart van Baalen, Tijmen Blankevoort, and Max Welling. Data-free quantization through weight equalization and bias correction. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 1325–1334, 2019.
- [21] Yury Nahshan, Brian Chmiel, Chaim Baskin, Evgenii Zheltonozhskii, Ron Banner, Alex M Bronstein, and Avi Mendelson. Loss aware post-training quantization. *Machine Learning*, 110(11-12):3245–3262, 2021.

- [22] Eunhyeok Park, Dongyoung Kim, and Sungjoo Yoo. Energy-efficient neural network accelerator based on outlier-aware low-precision computation. In *2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA)*, pages 688–698. IEEE, 2018.
- [23] Eunhyeok Park, Sungjoo Yoo, and Peter Vajda. Value-aware quantization for training and inference of neural networks. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 580–595, 2018.
- [24] Gunho Park, Baeseong Park, Minsub Kim, Sungjae Lee, Jeonghoon Kim, Beomseok Kwon, Se Jung Kwon, Byeongwook Kim, Youngjoo Lee, and Dongsoo Lee. Lut-gemm: Quantized matrix multiplication based on luts for efficient inference in large-scale generative language models, 2023.
- [25] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems*, 32, 2019.
- [26] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019.
- [27] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *The Journal of Machine Learning Research*, 21(1):5485–5551, 2020.
- [28] Teven Le Scao, Angela Fan, Christopher Akiki, Ellie Pavlick, Suzana Ilić, Daniel Hesslow, Roman Castagné, Alexandra Sasha Luccioni, François Yvon, Matthias Gallé, et al. Bloom: A 176b-parameter open-access multilingual language model. *arXiv preprint arXiv:2211.05100*, 2022.
- [29] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.
- [30] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Ed Chi, Quoc Le, and Denny Zhou. Chain of thought prompting elicits reasoning in large language models. *arXiv preprint arXiv:2201.11903*, 2022.
- [31] Xiuying Wei, Ruihao Gong, Yuhang Li, Xianglong Liu, and Fengwei Yu. Qdrop: Randomly dropping quantization for extremely low-bit post-training quantization. In *International Conference on Learning Representations*, 2022.
- [32] Xiuying Wei, Yunchen Zhang, Xiangguo Zhang, Ruihao Gong, Shanghang Zhang, Qi Zhang, Fengwei Yu, and Xianglong Liu. Outlier suppression: Pushing the limit of low-bit transformer language models. In *Advances in Neural Information Processing Systems*, 2022.
- [33] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, et al. Huggingface’s transformers: State-of-the-art natural language processing. *arXiv preprint arXiv:1910.03771*, 2019.
- [34] Guangxuan Xiao, Ji Lin, Mickael Seznec, Julien Demouth, and Song Han. Smoothquant: Accurate and efficient post-training quantization for large language models. *arXiv preprint arXiv:2211.10438*, 2022.
- [35] Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Victoria Lin, et al. Opt: Open pre-trained transformer language models. *arXiv preprint arXiv:2205.01068*, 2022.
- [36] Shuchang Zhou, Yuxin Wu, Zekun Ni, Xinyu Zhou, He Wen, and Yuheng Zou. Dorefa-net: Training low bitwidth convolutional neural networks with low bitwidth gradients. *arXiv preprint arXiv:1606.06160*, 2016.

A Main Proofs

1 Proof of Eq. (5) in the Main Manuscript

Our goal is to find the quantized weight $\hat{W}$ that minimizes the objective function:

$$\arg \min_{\hat{W}} E = \arg \min_{\hat{W}} \|WX - \hat{W}X\|_2^2 \quad \text{s.t.} \quad C(\hat{W}) < C_t. \quad (6)$$

we can reorganize the squared error term in Eq. (6) as the sum of squared errors corresponding to each output channel in the weight matrix:

$$E = \sum_{i=1}^{C_{out}} \|W_{i,:}X - \hat{W}_{i,:}X\|_2^2 = \sum_{i=1}^{C_{out}} E_i. \quad (7)$$

From this decomposition, we can see that the overall error can be separated into the sum of the errors from each output channel. Therefore, our goal of minimizing overall error E can be thought of as minimizing the error of each output channel E_i . The error that occurs when quantizing a network can be approximated by a Taylor series expansion for the model weights as follows:

$$E_i = \frac{\partial E_i}{\partial W_{i,:}} \Delta W_{i,:}^T + \frac{1}{2} \Delta W_{i,:} H_i \Delta W_{i,:}^T, \quad (8)$$

where $\Delta W_{i,:} = W_{i,:} - \hat{W}_{i,:}$ is the perturbation of the i 'th row of the weight and $H_i = \partial^2 E_i / \partial W_{i,:}^2$ is the Hessian matrix containing the second order derivatives for all weights in the weight row $W_{i,:}$. Since E_i is a quadratic function, all terms above the third order become zero. For a network trained with a local minimum of error, the first-order (gradient) term can be ignored:

$$E_i \approx \Delta W_{i,:} H_i \Delta W_{i,:}^T. \quad (9)$$

Therefore, the quantization error for each output channel can be approximated as Eq. (9).

B Experiment Details

The implementation of OWQ is based on OPTQ (GPTQ) official GitHub [14].

1 Evaluation Settings

We measured language modeling perplexity on datasets from WikiText-2 [17], PTB [16], and C4 [27]. The validation set is concatenated using two newlines as separators in WikiText-2 and a space as a separator in the PTB and C4 and then the concatenated data is tokenized using the default HuggingFace [33] tokenizer for each model.

2 Score Issues on LLaMA Model Family

In Section C, the PTB perplexity of the LLaMA [29] model is poor. This is a problem of a full-precision model, not due to quantization. Another issue is that the accuracy we obtained in our zero-shot experiments using EleutherAI/lm-evaluation-harness [11] is slightly lower than that published in the LLaMA paper. We expect the accuracy to be different due to differences in the experimental environment or special tokens. Since all experiments were conducted in the same environment, we can compare the results regardless of these issues.

3 Effective Bit-Width

In Section 4 of the main manuscript, we described that we store a complete low-precision matrix with zero-filled weak columns and additional fp16 weak columns (latency-favored method). Another possible option is storing the reduced size low-precision matrix and fp16 weak columns to keep the total number of weight elements the same and to avoid storing unnecessary zeros corresponding to weak columns (storage-favored method). The latter method has trade-offs: it can save more storage, but it has more overhead in the actual operation.

We calculated the effective bit-width using the storage-favored method for the accuracy results in the main manuscript and the supplementary materials. However, the memory overhead is similar for both methods as there are few weak columns.

If we add the storage overhead of the zero-filled low-precision matrix to our 3.01-bit case, it becomes 3.012-bit, which is negligible overhead.

C Additional Results

In this section, the results with * mark in the tables used 3.05-bit. Otherwise, there are few or no weak columns in the budget of 3.01-bit due to the small model dimension. Similar to the OPTQ, the calibration data were sampled from the C4 training set, so measuring perplexity on C4 is not a fully zero-shot situation.

1 BLOOM Model Results

We additionally checked the effectiveness of the proposed OWQ on BLOOM [28] model families. Table 9, Table 10, and Table 11 show perplexity results on BLOOM models. For the 176B case, we used a single seed.

Table 9: BLOOM WikiText-2 perplexity (lower is better).

LLaMA	Bits	560M	1.1B	1.7B	3B	7.1B	176B
full	16	22.41	17.68	15.39	13.48	11.37	8.11
RTN	4	25.89	19.99	16.97	14.75	12.10	8.37
OPTQ	4	24.06	18.92	16.35	14.11	11.75	8.19
OWQ	4.01	23.25$\pm$.06	18.20$\pm$.04	15.70$\pm$.01	13.72$\pm$.01	11.50$\pm$.01	8.14
OPTQ	3	32.51	25.07	21.33	17.53	13.72	8.63
OWQ	3.01	24.70$\pm$.08	19.56$\pm$.02	16.70$\pm$.05	14.40$\pm$.05	11.92$\pm$.02	8.26
OWQ	3.1	24.53$\pm$.06	19.35$\pm$.07	16.52$\pm$.05	14.34$\pm$.02	11.85$\pm$.02	8.25

Table 10: BLOOM Penn Treebank (PTB) perplexity.

LLaMA	Bits	560M	1.1B	1.7B	3B	7.1B	176B
full	16	43.67	57.98	29.99	25.33	20.82	14.59
RTN	4	51.07	66.83	33.56	27.67	22.41	15.00
OPTQ	4	47.20	63.25	32.17	26.50	21.69	14.71
OWQ	4.01	45.68$\pm$.29	60.04$\pm$.20	30.79$\pm$.07	25.84$\pm$.07	21.15$\pm$.04	14.61
OPTQ	3	67.08	87.81	44.91	34.85	26.49	15.47
OWQ	3.01	48.38$\pm$.25	67.01$\pm$.46	33.64$\pm$.21	27.84$\pm$.12	22.11$\pm$.04	14.88
OWQ	3.1	47.81$\pm$.33	64.52$\pm$.59	32.43$\pm$.21	27.36$\pm$.12	21.86$\pm$.04	14.88

Table 11: BLOOM C4 perplexity.

LLaMA	Bits	560M	1.1B	1.7B	3B	7.1B	176B
full	16	26.59	22.05	19.49	17.48	15.20	11.71
RTN	4	29.88	24.43	21.25	18.76	16.05	12.04
OPTQ	4	28.02	23.20	20.56	18.07	15.59	11.79
OWQ	4.01	$28.11 \pm .23$	$22.51 \pm .01$	$19.83 \pm .01$	$17.72 \pm .00$	$15.35 \pm .01$	11.73
OPTQ	3	35.68	28.78	25.20	21.22	17.60	12.24
OWQ	3.01	$28.93^* \pm .03$	$23.83 \pm .01$	$20.81 \pm .01$	$18.42 \pm .01$	$15.82 \pm .00$	11.85
OWQ	3.1	$28.87 \pm .03$	$23.60 \pm .02$	$20.61 \pm .01$	$18.31 \pm .01$	$15.75 \pm .00$	11.84

2 Additional Perplexity Results

Tables in this section (Table 12,13,14, and 15) show additional language generation task results.

Table 12: OPT Penn Treebank (PTB) perplexity.

OPT	Bits	125M	350M	1.3B	2.7B	6.7B	13B	30B	66B
full	16	38.99	31.08	20.29	17.97	15.77	14.52	14.04	13.36
RTN	4	53.89	36.79	57.30	31.05	18.84	16.51	15.40	225.66
OPTQ	4	45.76	34.26	22.06	19.13	16.54	14.86	14.22	13.77
OWQ	4.01	$41.89 \pm .61$	$32.47 \pm .22$	$21.35 \pm .16$	$18.45 \pm .05$	$15.79 \pm .04$	$14.77 \pm .02$	$14.14 \pm .01$	$13.43 \pm .01$
OPTQ	3	74.60	48.28	31.36	25.09	22.49	16.55	15.37	28.25
OWQ	3.01	$49.53^* \pm .77$	$37.68^* \pm .30$	$23.80 \pm .15$	$19.85 \pm .07$	$16.32 \pm .04$	$17.00 \pm .05$	$14.39 \pm .04$	$13.66 \pm .03$
OWQ	3.1	$46.98 \pm .95$	$37.26 \pm .43$	$22.27 \pm .13$	$19.35 \pm .07$	$16.30 \pm .04$	$14.98 \pm .03$	$14.33 \pm .05$	$13.62 \pm .02$

Table 13: LLaMA Penn Treebank (PTB) perplexity.

LLaMA	Bits	7B	13B	30B	65B
full	16	41.15	28.10	23.51	25.04
RTN	4	48.66	29.45	25.49	27.97
OPTQ	4	58.57	30.46	24.86	27.35
OWQ	4.01	$44.30 \pm .26$	$29.43 \pm .24$	$23.90 \pm .13$	23.62 ± 1.91
OPTQ	3	260.36	47.49	29.99	29.64
OWQ	3.01	55.60 ± 1.52	$31.39 \pm .40$	$24.75 \pm .16$	$24.21 \pm .20$
OWQ	3.1	53.82 ± 1.81	$30.73 \pm .17$	$24.46 \pm .18$	$24.45 \pm .31$

Table 14: OPT C4 perplexity.

OPT	Bits	125M	350M	1.3B	2.7B	6.7B	13B	30B	66B
full	16	26.56	22.59	16.07	14.34	12.71	12.06	11.44	10.99
RTN	4	33.91	26.21	24.51	18.43	14.36	13.36	13.46	309.
OPTQ	4	29.32	24.68	16.96	14.97	13.17	12.25	11.57	11.22
OWQ	4.01	$27.92 \pm .01$	$23.36 \pm .02$	$16.49 \pm .01$	$14.60 \pm .01$	$12.83 \pm .00$	$12.17 \pm .01$	$11.49 \pm .00$	$11.02 \pm .00$
OPTQ	3	42.11	31.50	21.53	18.10	17.29	13.33	12.22	14.07
OWQ	3.01	$31.30^* \pm .05$	$26.32^* \pm .10$	$17.69 \pm .02$	$15.35 \pm .01$	$13.23 \pm .01$	$13.31 \pm .02$	$11.69 \pm .00$	$11.16 \pm .00$
OWQ	3.1	$30.03 \pm .05$	$25.94 \pm .06$	$17.13 \pm .01$	$15.16 \pm .01$	$13.18 \pm .01$	$12.42 \pm .00$	$11.68 \pm .01$	$11.16 \pm .00$

Table 15: LLaMA C4 perplexity.

LLaMA	Bits	7B	13B	30B	65B
full	16	7.34	6.80	6.13	5.81
RTN	4	8.12	7.23	6.54	6.07
OPTQ	4	8.77	7.09	6.41	6.02
OWQ	4.01	$7.66 \pm .01$	$6.97 \pm .00$	$6.25 \pm .00$	$5.95 \pm .00$
OPTQ	3	21.77	8.91	7.94	7.16
OWQ	3.01	$8.62 \pm .04$	$7.44 \pm .01$	$6.73 \pm .01$	$6.28 \pm .01$
OWQ	3.1	$8.15 \pm .02$	$7.31 \pm .01$	$6.57 \pm .00$	$6.16 \pm .00$

3 Additional Zero-shot Results

Tables in this section (Table 16, 17, 18, 19, and 20) show additional results for several zero-shot benchmarks: PIQA [1], ARC-easy and ARC-challenge [3], OpenBookQA [18].

Table 16: OPT PIQA zero-shot accuracy (%) (higher is better).

OPT	Bits	125M	350M	1.3B	2.7B	6.7B	13B	30B	66B
full	16	62.02	64.69	72.47	74.76	76.55	76.77	78.13	79.87
RTN	4	61.53	63.44	67.57	73.67	76.39	76.12	77.26	60.12
OPTQ	4	61.59	64.18	70.84	73.93	76.25	76.63	78.54	78.91
OWQ	4.01	61.83 $\pm_{.15}$	64.94 $\pm_{.21}$	71.52 $\pm_{.60}$	74.34 $\pm_{.17}$	76.67 $\pm_{.14}$	76.50 $\pm_{.11}$	78.48 $\pm_{.34}$	79.54 $\pm_{.14}$
OPTQ	3	58.78	62.46	68.43	70.62	73.05	74.70	77.76	70.63
OWQ	3.01	60.51 $\pm_{.20}$	63.33 $\pm_{.24}$	70.47 $\pm_{.25}$	73.60 $\pm_{.44}$	75.85 $\pm_{.45}$	75.40 $\pm_{.16}$	78.43 $\pm_{.46}$	79.28 $\pm_{.26}$
OWQ	3.1	60.53 $\pm_{.27}$	63.42 $\pm_{.58}$	70.83 $\pm_{.28}$	73.69 $\pm_{.32}$	76.22 $\pm_{.22}$	76.04 $\pm_{.08}$	78.20 $\pm_{.31}$	79.28 $\pm_{.12}$

Table 17: OPT ARC-easy zero-shot accuracy (%).

OPT	Bits	125M	350M	1.3B	2.7B	6.7B	13B	30B	66B
full	16	39.86	40.45	50.97	54.29	60.06	61.83	65.32	67.26
RTN	4	36.24	38.59	42.72	52.90	58.67	61.32	61.20	40.49
OPTQ	4	38.40	38.65	49.61	52.54	58.95	61.16	64.59	64.91
OWQ	4.01	38.17 $\pm_{.37}$	39.35 $\pm_{.29}$	49.28 $\pm_{.21}$	54.11 $\pm_{.28}$	59.56 $\pm_{.33}$	61.85 $\pm_{.37}$	64.76 $\pm_{.16}$	66.82 $\pm_{.13}$
OPTQ	3	36.33	36.70	44.85	48.43	53.75	56.56	61.34	49.32
OWQ	3.01	36.70 $\pm_{.67}$	37.68 $\pm_{.46}$	47.14 $\pm_{.38}$	52.06 $\pm_{.50}$	57.68 $\pm_{.30}$	60.03 $\pm_{.50}$	63.34 $\pm_{.53}$	66.21 $\pm_{.18}$
OWQ	3.1	37.26 $\pm_{.19}$	37.92 $\pm_{.50}$	49.25 $\pm_{.23}$	52.70 $\pm_{.18}$	58.01 $\pm_{.81}$	60.91 $\pm_{.49}$	63.70 $\pm_{.39}$	66.30 $\pm_{.25}$

Table 18: LLaMA PIQA (left) and ARC-easy (right) zero-shot accuracy (%).

LLaMA	Bits	PIQA				ARC-easy			
		7B	13B	30B	65B	7B	13B	30B	65B
full	16	77.37	79.11	80.14	80.85	52.48	59.85	58.92	58.80
RTN	4	76.17	77.91	79.27	80.41	52.23	57.07	60.61	58.46
OPTQ	4	75.36	78.12	79.68	80.83	51.73	57.99	58.23	58.92
OWQ	4.01	77.08 $\pm_{.15}$	78.59 $\pm_{.07}$	80.58 $\pm_{.19}$	80.12 $\pm_{.22}$	50.76 $\pm_{.60}$	58.50 $\pm_{.41}$	58.54 $\pm_{.48}$	57.60 $\pm_{.66}$
OPTQ	3	59.78	75.24	77.23	78.69	33.90	53.32	51.36	54.95
OWQ	3.01	75.71 $\pm_{.40}$	77.63 $\pm_{.47}$	79.61 $\pm_{.33}$	78.66 $\pm_{.42}$	47.42 $\pm_{.72}$	54.18 $\pm_{1.01}$	57.62 $\pm_{1.27}$	53.38 $\pm_{.60}$
OWQ	3.1	76.65 $\pm_{.20}$	78.16 $\pm_{.45}$	79.43 $\pm_{.30}$	80.10 $\pm_{.56}$	49.82 $\pm_{1.01}$	56.48 $\pm_{.74}$	56.88 $\pm_{.86}$	56.71 $\pm_{1.15}$

Table 19: OPT ARC-challenge zero-shot accuracy (%).

OPT	Bits	125M	350M	1.3B	2.7B	6.7B	13B	30B	66B
full	16	22.87	23.89	29.52	31.23	34.56	35.75	38.05	40.02
RTN	4	22.35	23.72	24.91	29.10	32.85	35.32	35.58	22.87
OPTQ	4	22.59	23.84	29.04	29.61	33.12	35.31	36.96	37.58
OWQ	4.01	22.39 $\pm_{.43}$	24.10 $\pm_{.61}$	28.79 $\pm_{.42}$	30.41 $\pm_{.34}$	34.04 $\pm_{.53}$	35.38 $\pm_{.44}$	37.65 $\pm_{.61}$	39.87 $\pm_{.40}$
OPTQ	3	22.10	23.23	26.52	28.41	30.73	32.94	35.36	29.98
OWQ	3.01	22.56 $\pm_{.65}$	23.60 $\pm_{.34}$	28.07 $\pm_{.32}$	29.64 $\pm_{.73}$	32.41 $\pm_{.37}$	35.32 $\pm_{.60}$	36.76 $\pm_{.46}$	39.13 $\pm_{.65}$
OWQ	3.1	23.02 $\pm_{.13}$	23.70 $\pm_{.51}$	28.38 $\pm_{.73}$	29.20 $\pm_{.62}$	33.19 $\pm_{.58}$	34.74 $\pm_{.54}$	36.66 $\pm_{.22}$	39.16 $\pm_{.59}$

Table 20: OPT OpenBookQA zero-shot accuracy (%).

OPT	Bits	125M	350M	1.3B	2.7B	6.7B	13B	30B	66B
full	16	28.0	28.2	33.0	35.2	37.4	39.0	40.2	41.0
RTN	4	27.0	27.8	29.6	36.4	37.8	36.6	38.2	27.8
OPTQ	4	26.40	27.32	33.56	35.08	36.72	37.92	39.52	41.04
OWQ	4.01	28.64 ± 0.55	27.48 ± 0.87	32.28 ± 0.30	35.24 ± 0.17	37.84 ± 0.26	38.84 ± 0.74	40.32 ± 0.66	41.00 ± 1.4
OPTQ	3	27.64	26.48	29.08	32.36	35.92	36.24	38.68	31.08
OWQ	3.01	27.68 ± 0.88	26.72 ± 0.76	30.68 ± 1.27	32.92 ± 0.58	37.00 ± 0.71	37.84 ± 1.02	38.44 ± 0.46	40.40 ± 0.24
OWQ	3.1	27.88 ± 1.22	27.20 ± 0.79	32.52 ± 0.36	34.24 ± 0.78	37.68 ± 0.95	37.92 ± 0.30	38.80 ± 0.37	40.32 ± 0.30

4 Language Generation Results

We analyzed the linguistic abilities of the quantized model through generated outputs by prompting the model. We experimented with full-precision (FP16) LLaMA 7B, 30B model, 3-bit quantized 30B model with OPTQ, and 3.01-bit quantized 30B model using OWQ. Fig. 4 shows the input prompts and the corresponding outputs for each model. The $n / 10$ notation at the upper-right of each output box indicates the number of correct answers out of 10 total attempts. Each output example in Fig. 4 has been randomly selected among 10 attempts. In our experiments, we used some prompt examples from the Chain-of-Thought Prompting [30]. Fig. 4 shows that the 30B OWQ 3.01-bit model has comparable linguistic abilities to the full-precision 30B model, while having a similar storage capacity to the full-precision 7B model.

Prompt	Output
Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now? A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11. Q: The cafeteria had 23 apples. If they used 5 to make lunch and bought 12 more, how many apples do they have? A:	LLaMA 7B / FP16 2 / 10 23 - 5 = 18. 18 - 12 = 6. 23 + 6 = 29. ❌ LLaMA 30B / FP16 8 / 10 23 - 5 = 18. 18 + 12 = 30. The answer is 30. ✅ LLaMA 30B / OPTQ 3-bit 1 / 10 23 + 5 + 12 = 37. They have 37 apples. ❌ LLaMA 30B / OWQ 3.01-bit 7 / 10 23 minus 5 is 18. 18 plus 12 is 30. The answer is 30. ✅
Prompt	Output
Q: The fox walked from the city into the forest, what was it looking for? Answer Choices: (a) pretty flowers (b) hen house (c) natural habitat (d) storybook A: The answer must be something in the forest. Of the above choices, only natural habitat is in the forest. So the answer is (b). Q: Where are you likely to find a hamburger? Answer Choices: (a) fast food restaurant (b) pizza (c) ground up dead cows (d) mouth (e) cow carcass A:	LLaMA 7B / FP16 0 / 10 4. The answer must be something that is dead, since a hamburger is made from beef, which is dead meat. ❌ LLaMA 30B / FP16 6 / 10 100% of the time, you are going to find a hamburger in a fast food restaurant. ✅ LLaMA 30B / OPTQ 3-bit 0 / 10 1, 2, and 3 are in the city. 4 is on the ground. 5 is in a mouth. The answer is (e). ❌ LLaMA 30B / OWQ 3.01-bit 7 / 10 99% of the time, a hamburger is found in a restaurant. So the answer is (a). ✅

Figure 4: The generation results of each model using the prompts for the math word problem (left) and commonsense problem (CSQA, right) tasks. We omitted the newlines between the answer choices for the CSQA prompt in this figure.

5 True Sequential and Activation Order

As mentioned in Section 6 of the main manuscript, the "act-order" option and "true-sequential" option were recently added to the official Github of OPTQ [14]. The "true-sequential" option allows sequential quantization even within a transformer block, while the "act-order" determines the quantization order based on activation magnitude.

These options are beneficial for OPTQ in general, especially for LLaMA models (Table 21, Table 22). However, there is no significant performance improvement for OWQ with these options.

Table 21: Perplexity results with true-sequential (TS) and act-order (AO) options on OPT models.

OPT	Tasks	OPT-6.7b				OPT-30b			
		-	TS	AO	TS+AO	-	TS	AO	TS+AO
OPTQ 3-bit	WikiText2	15.09	15.35	12.98	12.83	10.30	10.34	10.28	10.32
	PTB	22.49	22.64	19.26	19.37	15.37	15.34	15.13	15.11
	C4	17.29	17.57	14.58	14.55	12.22	12.21	12.14	12.14
OWQ 3.01 bit	WikiText2	11.22	11.23	11.23	11.24	9.58	9.62	9.64	9.59
	PTB	16.32	16.33	16.41	16.35	14.39	14.39	14.41	14.41
	C4	13.23	13.23	13.22	13.22	11.69	11.69	11.69	11.69

Table 22: Perplexity results with true-sequential (TS) and act-order (AO) options on LLaMA models.

LLaMA	Tasks	LLaMA 7B				LLaMA 30B			
		-	TS	AO	TS+AO	-	TS	AO	TS+AO
OPTQ 3-bit	WikiText2	18.07	18.11	8.18	8.09	5.84	5.77	5.72	5.66
	PTB	260.36	199.19	73.89	70.47	29.99	30.02	29.41	28.45
	C4	21.77	20.77	10.35	10.27	7.94	7.88	7.89	7.81
OWQ 3.01 bit	WikiText2	6.65	6.65	6.56	6.52	4.76	4.77	4.71	4.70
	PTB	55.60	57.49	52.03	55.26	24.75	24.76	24.55	24.59
	C4	8.62	8.62	8.38	8.39	6.73	6.73	6.69	6.70